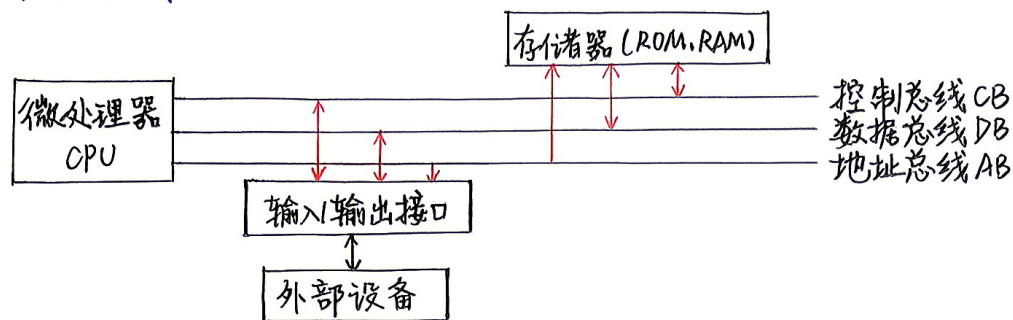


第1章

微型计算机的系统组成



掌握: 微机 IBM PC 系列机 (第3代 - 16位微机 - 8086 → 8088 → 80286)

汇编语言程序设计方法

微型计算机的接口技术 { 半导体存储器及其接口 { 硬件: 接口电路原理
I/O 接口电路及其与外设连接 { 软件: 接口编程方法

1946年, 世界上出现了第一台数字式电子计算机 ENIAC (电子数据和计算器)

发展到以大规模集成电路为主要部件的第4代, 产生了微型计算机

以大规模、超大规模集成电路为主要部件,

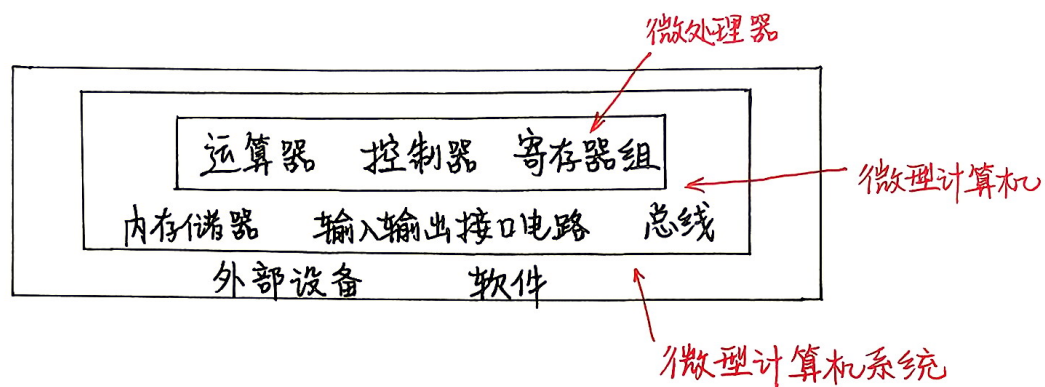
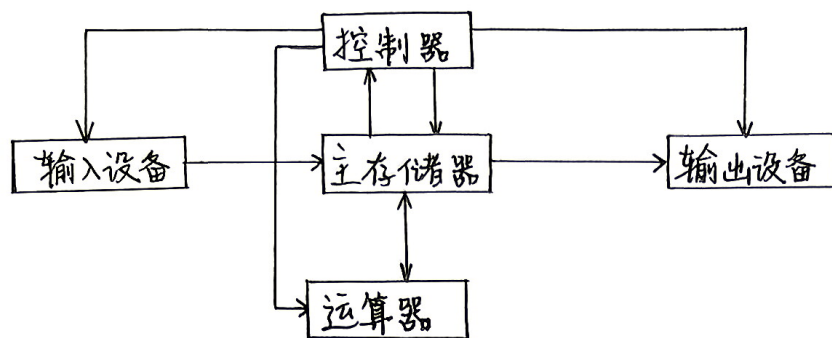
以集成控制器和运算器(计算机主要部件)的微处理器为核心,

构造出的计算机系统

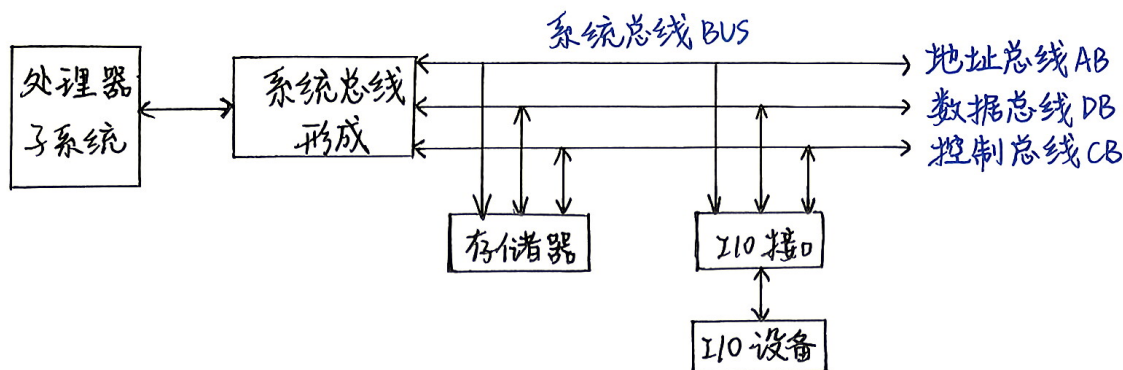
1971年, Intel 公司设计了世界上第一个微处理器芯片 Intel 4004

型号	发布年份	字长(位)	数据总线宽度(位)	外部总线宽度(位)	地址总线宽度(位)	寻址空间(B)	高速缓存
8086	1978	16	16	16	20	1M	无
8088	1979	16	16? 8?	8	20	1M	无

从ENIAC到当前最先进的计算机,基本结构均属于冯·诺依曼型计算机



微型计算机的硬件组成



总线是指传递信息的一组公用导线，微机系统采用总线结构连接系统功能部件
总线接口也是一种通用的接口技术

① 地址总线 AB

作用：输出将要访问的内存单元或 I/O 端口的地址

决定系统直接寻址存储器的范围

② 数据总线 DB

作用：CPU 读操作时外部数据通过数据总线送往 CPU；

CPU 写操作时 CPU 数据通过数据总线送往外部

决定一次能够传送数据的位数

③ 控制总线 CB

作用：协调各部件操作

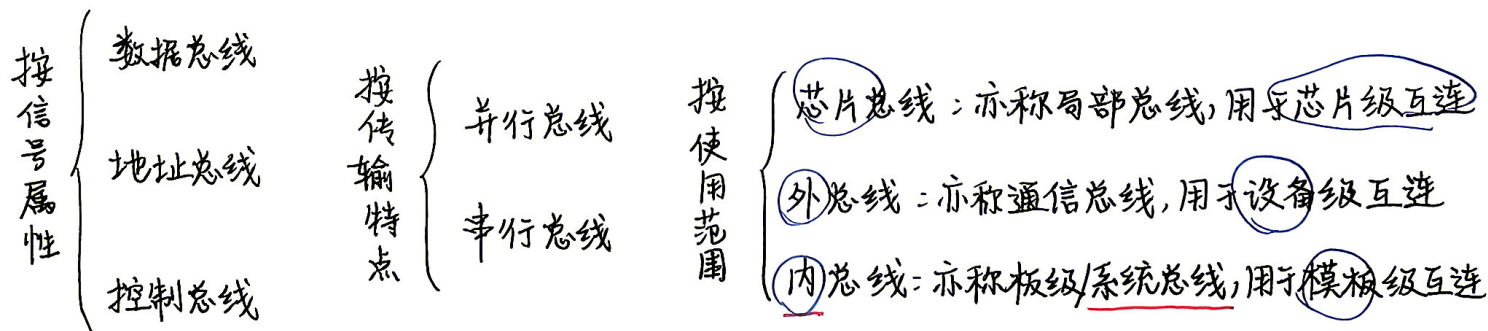
决定系统总线的特点（功能、适应性）

★ 某时刻只能由 1 个总线主控设备控制总线

★ 连接系统总线的各个设备中某时刻只能有 1 个发送者向总线发送信号，但可以有多个设备从总线上同时获取信号

微机系统采用“总线结构”，优点 = 组态灵活、扩展方便

总线的分类



微型计算机的软件系统

操作系统	MS-DOS
汇编程序	MASM, LINK
文本编辑程序	EDIT.COM
调试程序	DEBUG.EXE

十进制数转换成二进制数

例. $117.8125D = 1110101.1101B$

2	117	
2	58	1
2	29	0
2	14	1
2	7	0
2	3	1
2	1	1
	0	1

除2取余 (除到余数为0)

	0.8125	
x	2	
	1.6250	1
	0.625	
x	2	
	1.250	1
	0.25	
x	2	
	0.50	0
	0.50	
x	2	
	1.00	1

乘2取整 (乘到乘积小数部分为0)

十进制数转换成十六进制数

$$(2347)_{10} = (92B)_{16}$$

16	2347	余数
16	146	14 B
16	9	9
	0	

$$(0.6875)_{10} = (0.B)_{16}$$

	0.6875	整数部分
x	16	
	11.0000	11 B

二进制数转换成八进制数

$$(11010.11)_2 = (32.6)_8$$

$$\underbrace{011}_3 \underbrace{010}_2 \underbrace{0.110}_3$$

八进制数转换成二进制数

$$(34.6)_8 = (11100.11)_2$$

$$011100.110$$

二进制数转换成十六进制数

$$(11010110111111)_2 = (35BF)_{16}$$

$$\underbrace{0011}_3 \underbrace{0101}_4 \underbrace{1011}_4 \underbrace{1111}_4$$

$$(1111101.0101111)_2 = (7D.5E)_{16}$$

$$\underbrace{0111}_4 \underbrace{1101}_4 \underbrace{0101}_4 \underbrace{1110}_4$$

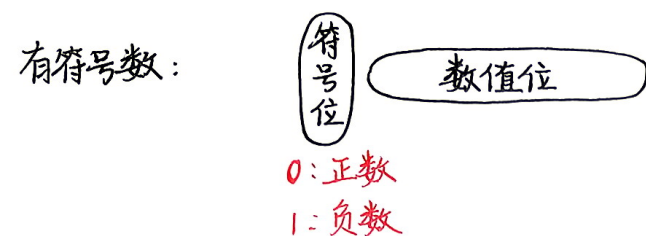
十六进制数转换成二进制数

$$(A19C)_{16} = (1010000110011100)_2$$

$$1010000110011100$$

{ 二进制 B
 八进制 O 或 Q
 十进制 D
 十六进制 H

计算机中数值型数据和非数值型数据 (例如字符) 都是用二进制数来编码表示的



假设机器字长是8位: 1位符号位, 7位数值位

原码

[+3] 原码 = 0 000,0011 = 03H	} 0 的表示不唯一
[-3] 原码 = 1 000,0011 = 83H	
[+0] 原码 = 0 000,0000 = 00H	
[-0] 原码 = 1 000,0000 = 80H	

反码

[+5] 反码 = 0 000,0101 = 05H	} 0 的表示不唯一
[-5] 反码 = 1 111,1010 = FAH	
[+0] 反码 = 0 000,0000 = 00H	
[-0] 反码 = 1 111,1111 = FFH	

正数的反码同原码;
负数的反码符号位是1, 数值位与原码相反
负数的反码是把符号位后的全部取过来.

补码

[+46] 补码 = 0010,1110 = 2EH	} 0 的表示唯一
[-46] 补码 = 1101,0010 = D2H	
[+0] 补码 = 0000,0000 = 00H	
[-0] 补码 = 0000,0000 = 00H	

正数的补码同原码;
负数的补码则是其对应正数的补码
"按位取反" "末位加1"
负数的补码是其反码+1.

有符号数
n位二进制补码的表示范围: $-2^{n-1} \leq N \leq 2^{n-1} - 1$

8位: $-128 \sim +127$

十进制	二进制	十六进制
+127	0111 1111	7F
+126	0111 1110	7E
⋮	⋮	⋮
+1	0000 0001	01
0	0000 0000	00
-1	1111 1111	FF
-2	1111 1110	FE
⋮	⋮	⋮
-126	1000 0010	82
-127	1000 0001	81
-128	1000 0000	80

除负的这个100...000 特殊
负数补码: -1. 取反.
得正数

$$[X+Y]_{\text{补码}} = [X]_{\text{补码}} + [Y]_{\text{补码}}$$

$$[X-Y]_{\text{补码}} = [X]_{\text{补码}} + [-Y]_{\text{补码}}$$

$$\begin{array}{r} 64 \\ + (-46) \\ \hline 18 \end{array}$$

$$\begin{array}{r} 01000000 \\ + 11010010 \\ \hline \cancel{1}00010010 \end{array}$$

进位: 由于运算结果超出了位数, 最高有效位向前的进位自然丢失

$$\begin{array}{r} (-64) \\ + 64 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 11000000 \\ + 01000000 \\ \hline \cancel{1}00000000 \\ 0 \quad \checkmark \text{进位} \end{array}$$

溢出: 表示结果超出字长允许的表示范围, 造成结果出错

$$\begin{array}{r} 127 \\ + 1 \\ \hline 128 \end{array}$$

$$\begin{array}{r} 01111111 \\ + 00000001 \\ \hline 10000000 \\ -128 \quad \times \text{溢出} \end{array}$$

无符号整数的表示范围: $0 \leq N \leq 2^n - 1$

常见的例如地址 / 双字长数据的低位字

计算机本身不论是对有符号数还是无符号数, 总是按照补码的运算规则做运算

十进制数的编码

BCD 码 (二-十进制编码):

8421 编码, 用4位二进制数表示1位十进制数, 自左到右每一位对应的位权分别是 8, 4, 2, 1

↑
压缩的用4位, 例如 4256: 0100 0010 0101 0110 B

非压缩的用8位, 例如 4256: xxxx0100 xxxx0010 xxxx0101 xxxx0110 B

字符编码

ASCII 码:

用1个字节表示1个字符, 最高位用作校验位, 低7位为字符的 ASCII 值

'A' 41H
'a' 61H
'1' 31H

换行 0AH
回车 0DH
空格 20H SP

ASCII 码共编码了 128 个字符

32 个控制字符, 十进制编码值 0~31

间隔字符 (空格字符) (20H)

删除控制码 DEL (7FH)

94 个可印刷字符 (有形字符)

0~9, 10 个数字字符 { 高3位: 011
低4位: 0000~1001
英文字母 { 小写: 次高位是 1
大写: 次高位是 0

汉字编码 一般用2个字节, 即16位二进制数表示

输入编码 (外码) 顺序码、音码、形码、音形码

此探讨不算
校验位在内

机内码 (内码) 2 个字节的最高位都是 1

字形码 (字模点阵码)

逻辑数据的编码

与 AND ^

或 OR V

非 NOT -

异或 XOR ∨

逻辑数据只能进行逻辑运算, 结果还是逻辑数据

例: $X = 00FFH$, $Y = 5555H$, 求 $Z = X \vee Y = ?$

$$\begin{array}{r} 0000 \ 0000 \ 1111 \ 1111 \ B \\ \vee \ 0101 \ 0101 \ 0101 \ 0101 \ B \\ \hline 0101 \ 0101 \ 1010 \ 1010 \ B \end{array}$$

$\therefore Z = 55AAH$

8086/8088 微处理器结构

对16位数据的存储器读/写操作要
2个总线周期才可完成



内部结构

8086/8088微处理器内部结构基本相同,区别在于8088的指令队列为4个字节,对外的数据总线是8位

8086 CPU 从功能上可以分为 { 总线接口单元 BIU 两部分

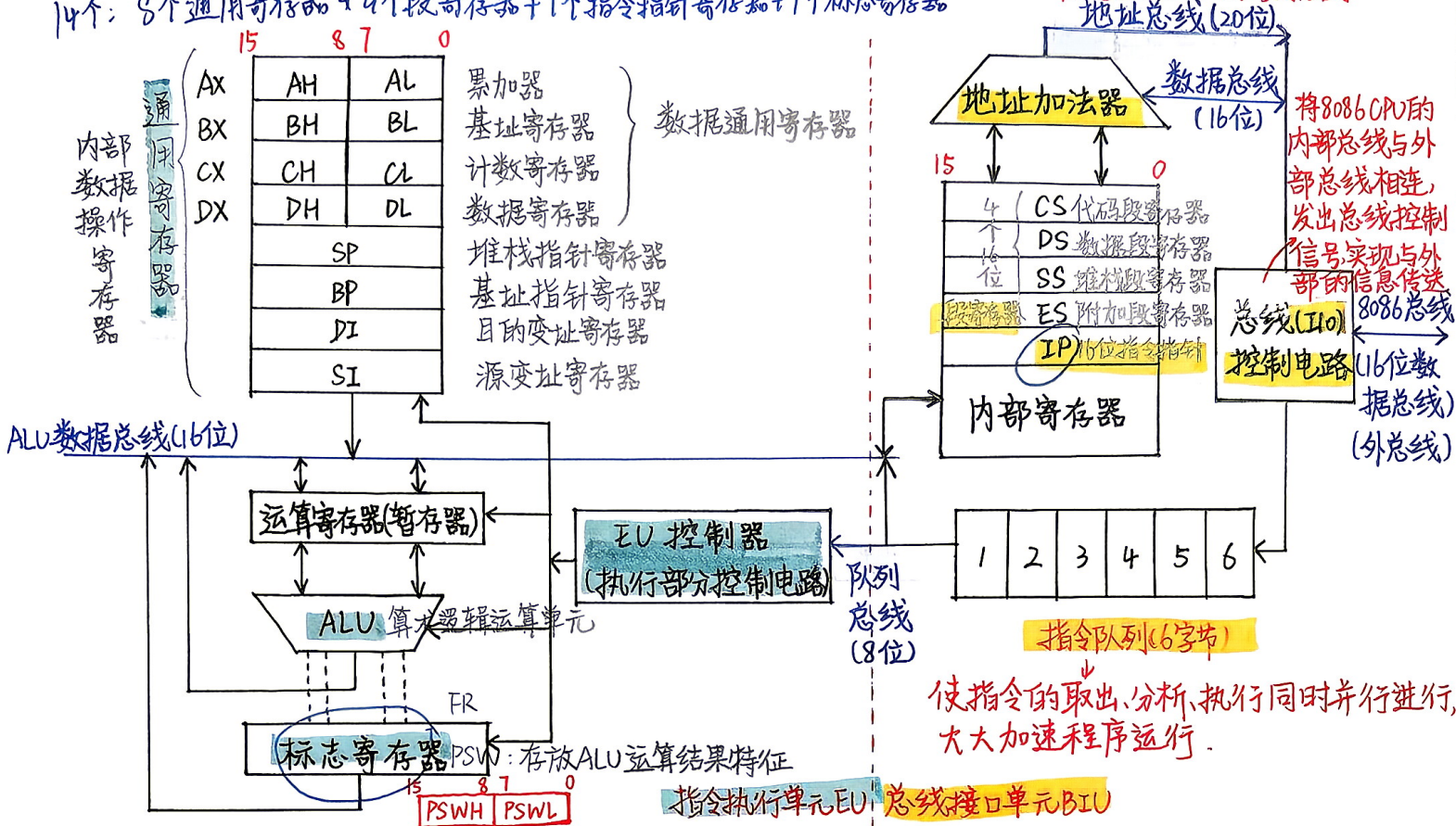
8086: 6-2

(8086 的是 16 位)

指令执行单元 EU

14个: 8个通用寄存器 + 4个段寄存器 + 1个指令指针寄存器 + 1个标志寄存器

8086: 20根地址线, 16根数据线



总线接口单元BIU的操作过程: 负责CPU与存储器, I/O端口传送数据

输入/输出控制电路将CPU内部总线与系统总线相连,是数据交换的必经之路

① 取指令：在CS中的16位段基址的最低位后面补4个0，加上IP中的16位偏移地址，

在地址加法器中形成20位物理地址直接送往地址总线,

通过总线控制逻辑发出存储器读信号,按给定地址从存储器中取出指令,送到指令队列中等待执行。

②指令队列：指令队列实际是内部的RAM阵列，先进先出，

最多可以保存6个指令字节,只要队列出现2个空字节且EU未要求BIU进入读写操作数的总线周期,BIU就自动从内存单元顺序取指令字节填满指令队列。

当执行转移指令时, BIU使指令队列^(清除)复位, 从新的地址单元取指令立即送往EU执行, 然后自动取出后继指令字节填满队列

③ 读/写操作数：EU从指令队列中取走指令，经指令译码，向BIU申请从存储器或I/O端口

读1字节操作数,只要收到EU送来的16位偏移地址,BIU就通过地址加法器将现行数据段及送来的偏移地址组成20位物理地址。

在当前取指令总线周期完成后,在读/写周期访问存储器或 I/O 端口完成读写操作。最后 EU 执行指令,由 BIU 将运算结果读出

指令执行单元 EU 的工作过程：

- ① EU 从 BIU 的指令队列输出端取出指令并进行分析，EU 根据指令要求向其内部各部件发控制命令，完成执行指令的功能
- ② 若指令需要访问总线取操作数，则 EU 将操作数的偏移地址通过 16 位数据总线送给 BIU，与段地址一起在 BIU 的地址加法器中形成 20 位物理地址，通过总线从内存中取操作数送给 EU
- ③ 指令若需算逻运算，EU 将运算的操作数取来存入暂存器（可从存储器或寄存器组中取），然后通过 ALU 进行各种算逻运算，结果由内部总线送到寄存器组或由 BIU 写入存储器或 I/O 端口。
运算所得结果的特征改变标志寄存器 PSW 的状态，供测试、判断、转移指令使用

EU 与 BIU 相互独立，分别完成各自操作，并非同步工作。

在 EU 执行指令的同时，BIU 可取下面一条或几条指令，

二者并行操作，加快系统的运行速度，提高工作效率。

内部寄存器

① 数据通用寄存器 (4个16位的: AX BX CX DX)

既用来存放源操作数, 又用来存放目标操作数和运算结果

AX (累加器): 使用频度最高, 用于算逻运算以及与外设传送信息

BX (基址寄存器): 存放存储器地址

CX (计数器): 循环或串操作等指令中的隐含计数器, 位操作中移多位时要用CL指明移位位数

DX (数据寄存器): 存放双字数据的高16位, 可作默认操作数参与乘除运算, 可存放外设端口地址

② 指针寄存器 (2个16位的: SP BP)

存放段内偏移地址, 与SS联用

SP (堆栈指针寄存器): 存放现行堆栈段的段内偏移地址 (栈顶)

BP (基址指针寄存器): 存放现行堆栈段内一个数据区的基地址偏移量 (某一存储单元)

③ 变址寄存器 (2个16位的: SI DI)

在字符串操作中存放源操作数的段内偏移量和目标操作数的段内偏移量

在串指令中隐含寻址, SI与DS联用, DI与ES联用。

SI (源变址寄存器): 存放源操作数的段内偏移量

DI (目的变址寄存器): 存放目标操作数的段内偏移量

④ 段寄存器 (4个16位的: CS DS SS ES) 数据的存放比较灵活, 其实可以存放在任何一种逻辑段中

8086把1M字节的存储空间划分为若干逻辑段, 每个逻辑段长度64K字节,

并规定每个逻辑段20位起始地址的最低4位是0000B, 仅高16位为有效数字 段的基地址

CS (代码段寄存器): 存放当前代码段的基地址, 代码段中存放要执行的指令 存放于段寄存器中
→ CS:IP 取得下一条要执行的指令 (CS放代码段的段地址, IP指示下一条指令的偏移地址)

DS (数据段寄存器): 存放当前数据段的基地址, 数据段中存放指令中所需操作数
→ DS:EA 存取数据段中的数据 (DS放数据段的段地址, EA即操作数的偏移地址)

SS (堆栈段寄存器): 存放当前堆栈段的基地址, 堆栈段存放堆栈段处理的数据
→ SS:SP 操作堆栈顶的数据 (SS放堆栈段的段地址, SP指示堆栈栈顶的偏移地址)

ES (附加段寄存器): 存放当前附加段的基地址, 附加段存放操作数
→ ES:EA 存取附加段中的数据 (操作数通常存于现行数据段, ES放附加段的段地址, EA即操作数偏移地址)

16位CPU系统中只有4个段寄存器, 32位微机和系统中有6个 串指令中目的操作数指明必在现行附加段

32位CPU有 { 实方式 两个工作方式
 保护方式

串操作指令将附加段作为其目的操作数的存放区域 ES

内存单元的逻辑地址仍为 "段值·偏移量" 形式

要访问某内存段内的数据, 必须使用该段寄存器和存储单元的偏移量

⑤ 指令指针寄存器 (1个16位的: IP)

IP (指令指针寄存器): 存放下一条要取出指令在现行代码段中的偏移地址

IP寄存器不能直接由程序员访问, 但可以通过某些指令修改IP的内容:

- 1) 跳转指令, 将新的跳转目标地址偏移量送入IP, 改变内容, 实现程序转移
- 2) 子程序指令, 8086自动调整IP内容, 将IP中下一条将要执行的指令地址偏移量入栈保护, 待子程序返回时将保护的内容从堆栈中弹出到IP, 使主程序继续运行

⑥ 标志寄存器 (9个16位的)

记录程序运行结果的状态信息

6个状态标志 CF PF AF ZF SF OF ,

3个控制标志 TF IF DF

控制处理器执行指令的方式

FLAGS/PSW

CF (进位标志): 本次运算中最高位产生进位(加法)或借位(减法)时, CF=1. 否则CF=0

PF (奇偶校验标志位): 本次运算结果低8位有偶数个“1”时, PF=1. 否则PF=0

例如 $3AH + 7CH = B6H = 10110110B \Rightarrow PF=0$

AF (辅助进位标志): 本次运算结果低4位向高4位有进位或借位时, AF=1. 否则AF=0
一般用在BCD码运算中, 判断是否需要+进制调整

例如

3AH	00111010H
+ 7CH	+ 01111100H
10110110H	

$\Rightarrow AF=1$

ZF (零标志位): 本次运算结果各位都为零时, ZF=1. 否则ZF=0

例如

84H	10000100H
+ 7CH	01111100H
10000000H	

$\Rightarrow ZF=1$

SF (符号标志位): 该标志位状态总与运算结果最高有效位状态相同,

反映带符号数运算结果的正负情况, 结果为负时, SF=1. 结果为正时, SF=0

例如 $3AH + 7CH = B6H$. 最高位是1 $\Rightarrow SF=1$.

$84H + 7CH = (1)00H$ 最高有效位是0 $\Rightarrow SF=0$

OF (溢出标志位): 带符号数加减运算的结果产生溢出时, OF=1. 否则, OF=0

字节运算结果范围: $-128 \sim +127$

字运算结果范围: $-32768 \sim +32767$

例如 $3AH + 7CH = B6H$ 溢出 $\Rightarrow OF=1$

AAH	10101010H
+ 7CH	01111100H
100100110H	

未溢出 $\Rightarrow OF=0$

(1)26H

TF (陷阱标志 / 单步操作标志): 用于控制单步中断. 当 $TF=1$ 时执行指令就产生单步中断
CPU 每执行一条指令就自动产生一个内部中断, 使处理器转去执行一个中断服务程序;
以便为用户提供该条指令执行后各寄存器的状况
单步中断用于程序调试过程中 (逐条指令调试程序)
 $TF=1$, 处理器单步执行指令. $TF=0$, 处理器正常工作

IF (中断标志位): 用于控制外部可屏蔽中断是否可被处理器响应

$IF=1$, 允许 CPU 响应可屏蔽中断. $IF=0$, 不允许响应

复位 IF - CLI 指令: $IF=0$

置位 IF - STI 指令: $IF=1$

DF (方向标志): 用于指定字符串处理指令的步进方向

$DF=1$, 以递减方式由高地址向低地址方向进行

$DF=0$, 以递增方式由低地址向高地址方向进行

复位 DF - CLD 指令: $DF=0$

置位 DF - STD 指令: $DF=1$

形成20位物理地址：在段寄存器的16位段基址的最低位后面补4个0，再加16位段内偏移地址，在地址加法器中形成20位物理地址（左移4位 $\Leftrightarrow \times 16 \Leftrightarrow \times 10H$ ）

例如，代码段寄存器CS存放当前代码段基址，IP指令指针寄存器存放下一条要执行的指令的段内偏移地址，其中CS=3400H，IP=00C5H

$$\begin{array}{r} 34000H \\ + 00C5H \\ \hline 340C5H \end{array} \rightarrow 20\text{位存储单元的寻址地址}$$

处理器利用CS:IP取得下一条要执行的指令

存放代码段的段地址 指示下一条指令的偏移地址

处理器利用SS:SP操作堆栈顶的数据

存放堆栈段的段地址 指示堆栈栈顶的偏移地址

处理器利用DS:EA存取数据段中的数据

存放数据段的段地址 有效地址
(存储器中操作数的偏移地址)

处理器利用ES:EA存取附加段中的数据

存放附加段的段地址 有效地址
(存储器中操作数的偏移地址)

程序的指令序列必须安排在代码段

程序使用的堆栈必须在堆栈段

程序中的数据默认安排在数据段，也常在附加段（尤其是串操作的目的区必须是附加段）

（数据的存放比较灵活，实际上可存放于任何一种逻辑段）

⑦ 段超越前缀指令

未指明时,一般的数据访问在DS段;使用BP访问主存,则在SS段

当要改变默认情况,需使用段超越前缀指令(8088指令系统有4个)

CS: : 代码段超越,使用代码段的数据
SS: : 堆栈段超越,使用堆栈段的数据
DS: : 数据段超越,使用数据段的数据
ES: : 附加段超越,使用附加段的数据

没有段超越的指令实例(从默认DS数据段取出数据):

MOV AX, [2000H] ; AX ← DS:[2000H]

采用段超越前缀的指令实例(从指定ES附加段取出数据):

MOV AX, ES:[2000H] ; AX ← ES:[2000H]

段寄存器的使用规定

访存方式	默认	可超越	偏移地址
取指令	CS	无	IP
堆栈操作	SS	无	SP
一般数据访问	DS	CS ES SS	有效地址EA
BP基址寻址方式	SS	CS DS ES	有效地址EA
串操作的源操作数	DS	CS ES SS	SI
串操作的目的操作数	ES	无	DI

小结 8088有8个8位通用寄存器,8个16位通用寄存器;6个状态标志,3个控制标志;
将1MB存储空间分段管理,4个段寄存器对应4种逻辑段;4个段超越前缀指令指定数据所在逻辑段
寄存器: 存储程序、数据等信息的记贮装置

SP: 保存当前栈顶地址的寄存器

控制标志: 控制操作的标志,如 DF、TF.

段地址: 逻辑段的起始地址

代码段: 存放指令机器码的存储器段区

△ 标号: 指出指令在程序中位置的符号地址

汇编程序: 将汇编语言程序翻译为机器码程序的实用程序

△ 助记符: 表示计算机执行何操作的符号